

Hier ist eine detaillierte Analyse des PDF-Auszugs, der eine Klausur mit verschiedenen Aufgaben zum

****Aufgabe 1: Fehlersuche (10 Punkte)****

****Ziel:**** Fehler in Python-Code-Snippets identifizieren und erklären.

**** (a) Fehler: Groß-/Kleinschreibung von `True` ****

- ****Fehler:**** `bestanden = true` (kleingeschriebenes `true`).
- ****Korrektur:**** `bestanden = True` (Python ist case-sensitive).
- ****Erklärung:**** `True` ist ein reserviertes Schlüsselwort in Python, während `true` eine ungültige Variable ist.

**** (b) Fehler: Längeninkonsistenz zwischen Listen ****

- ****Fehler:**** `stadt` hat 4 Elemente, `einwohner` nur 3.
- ****Korrektur:**** Anpassung der Listenlängen oder Fehlerbehandlung (z. B. `if i < len(einwohner)`).
- ****Erklärung:**** Der Code würde bei `i=3` einen `IndexError` werfen, da `einwohner[3]` nicht existiert.

**** (c) Fehler: Kein Basisfall in rekursiver Funktion ****

- ****Fehler:**** Keine Abbruchbedingung für `fakultaet(n)`.
- ****Korrektur:**** Hinzufügen von `if n == 0: return 1`.
- ****Erklärung:**** Ohne Basisfall führt die Rekursion zu einem Stack Overflow.

**** (d) Fehler: Uninitialisierte Variable `i` ****

- ****Fehler:**** `i` wird nicht definiert, bevor es in der `while`-Schleife verwendet wird.
- ****Korrektur:**** `i = 0` vor der Schleife.
- ****Erklärung:**** Python wirft einen `NameError`, da `i` nicht existiert.

**** (e) Fehler: Division durch Null ****

- ****Fehler:**** `b = 0.6` wird zu `int(b) = 0` konvertiert.
- ****Korrektur:**** `print(a / b)` (ohne `int()`).
- ****Erklärung:**** Division durch `0` ist mathematisch undefiniert und führt zu einer `ZeroDivisionError`.

****Aufgabe 2: Ausgabeanalyse (14 Punkte)****

****Ziel:**** Vorhersage der Ausgabe von Python-Code-Snippets.

**** (a) Ausgabe: `True` ****

- ****Code:**** `vier = 2`, `zwei = 4` → `vier\*vier == zwei` → `4 == 4` → `True`.

**** (b) Ausgabe: `3` ****

- ****Code:**** `range(3, 4)` → nur `i=3` wird gedruckt.

**** (c) Ausgabe: `i`, `j`, `k` ****

- ****Code:**** Schleife durchläuft die Liste `l` und druckt jedes Element.

**** (d) Ausgabe: `allesklar` ****

- ****Code:**** Indizes der Zeichen in `a` werden extrahiert und zu `a[11] + a[17] + ...` kombiniert.

**** (e) Ausgabe: `1`, `2`, `3`, `4`, `5` ****

- **Code:** Rekursive Funktion `k(i)` druckt `i`, bis `i > 5` ist.

Aufgabe 3: Run Length Encoding (32 Punkte)

Ziel: Implementierung von RLE-Komprimierung und -Dekomprimierung.

(a) `encode(text)`

- **Anforderung:** Komprimierung mit max. 9 aufeinanderfolgenden Zeichen.
- **Beispiel:** `AAAAAAAAAABBBCDDDDD` → `9A2A3B1C5D`.

(b) `decode(text)`

- **Anforderung:** Umkehrung der Komprimierung (z. B. `9A2A3B1C5D` → `AAAAAAAAAABBBCDDDDD`).

(c) Nachteile von RLE

- **Fälle, wo RLE ineffizient ist:**
 - Keine oder wenige Wiederholungen (z. B. `abcdef` → `1a1b1c1d1e1f`).
 - Lange Sequenzen mit unterschiedlichen Zeichen (z. B. `abcabcabc` → `3a3b3c`).

(d) `checkRLE(text)`

- **Anforderung:** Entscheidung, ob RLE sinnvoll ist (z. B. wenn mehr als 50% der Zeichen wiederholt).

Aufgabe 4: Schimpfwortfilter (30 Punkte)

Ziel: Zensur von Nachrichten basierend auf einer Schimpfwortliste.

(a) `readSwearwordData(filename)`

- **Anforderung:** Einlesen einer Datei mit Wörtern und Schweregraden (z. B. `Fuck 1` → `[["Fuck", 1]]`).

(b) `checkText(message, swearwords)`

- **Anforderung:**
 - Ersetzen von Schimpfwörtern durch `***`.
 - Löschen der Nachricht, wenn die Summe der Schweregrade > 7.
 - Groß-/Kleinschreibung ignorieren.

(c) Hauptprogramm

- **Beispiel:** `checkText("Du bist natürlich kein Idiot", readSwearwordData("swearwords.txt"))` → `"Du bist natürlich kein Idiot"`.

Aufgabe 5: Java-Klasse (14 Punkte)

Ziel: Implementierung einer Java-Klasse `Moebel` mit Konstruktor und `toString()`.

Lösung:

```
```java
public class Moebel {
 private String name;
 private double preis;
```

```

public Moebel(String name, double preis) {
 this.name = name;
 this.preis = preis;
}

@Override
public String toString() {
 return name + " (" + preis + " €)";
}

public static void main(String[] args) {
 Moebel regal = new Moebel("Yllib", 40);
 System.out.println(regal); // Ausgabe: "Yllib (40.0 €)"
}
}
...

```

---

#### ### \*\*Zusammenfassung\*\*

Die Klausur deckt grundlegende Python- und Java-Konzepte ab, darunter:

- Fehleranalyse und Debugging.
- String- und Listenoperationen.
- Rekursion und Schleifen.
- Dateioperationen und Textverarbeitung.
- Objektorientierte Programmierung (Java).

Die Aufgaben sind praxisorientiert und testen sowohl theoretisches Verständnis als auch praktische Im